

Introduction to Programming for Managerial Sciences

Dr. Öğr. Üyesi Fahrettin Çakır

February 13, 2026

Contents

1	Introduction to R	2
1.1	What is R?	2
1.2	Why Learn R?	2
1.3	The R Environment	2
1.4	R and Statistics	3
1.5	Using R and RStudio Interactively	6
1.6	Executing Scripts	7
1.7	Data Permanency and Removing Objects	8
1.8	Comments in R	9
1.9	Working Directory	10
1.10	Saving Work and Exiting R	11
1.11	Scripts in R	12
1.12	Exercise: Using the Help System in R	13
1.13	Exercise: Creating and Assigning Variables in R	14
1.14	Exercise: Removing Variables from the Environment	15
2	Understanding R Syntax	16
3	Basic Operators in R	17
3.1	Arithmetic, Comparison, and Logical Operators	17
4	Objects in R	18
4.1	Managing Objects in R	18
4.2	Types of R Objects (Data Structures)	19
4.3	Types of R Objects (Matrices)	20
4.4	Types of R Objects (Lists)	21
4.5	Types of R Objects (Data Frames)	23
4.6	Types of R Objects (Factors)	24
5	Objects, Their Modes, and Attributes in R	26
5.1	Changing the Length of an Object	28
5.2	Getting and Setting Attributes in R	29
5.3	The Class of an Object in R	30
5.4	Mode vs. Class in R	31
6	Vectors in R	32
6.1	Sequences, Repetition, Sorting, and Lengths in R	33
6.2	Sequences with <code>seq()</code> in R	34
6.3	Repetition with <code>rep()</code> in R	35
6.4	Sorting with <code>sort()</code> in R	36
6.5	Subsetting and Element Extraction in R	37
6.6	Removing Elements with Negative Indexing in R	38
6.7	Indexing with Vectors of Indexes in R	39
6.8	Modifying Elements in a Vector in R	41

1 Introduction to R

1.1 What is R?

R is a programming language and environment for statistical computing and graphics. It is:

- Open-source and widely used in data science, statistics, and machine learning.
- Maintained through the CRAN (Comprehensive R Archive Network).
- Developed by Ross Ihaka and Robert Gentleman in the early 1990s.

1.2 Why Learn R?

R is widely adopted due to:

- Free and open-source availability.
- Powerful data visualization and statistical capabilities.
- Active community support.
- Usage by companies such as Google, Facebook, and Airbnb.

1.3 The R Environment

R is an integrated suite of software facilities for data manipulation, calculation, and graphical display. Among other things, it has:

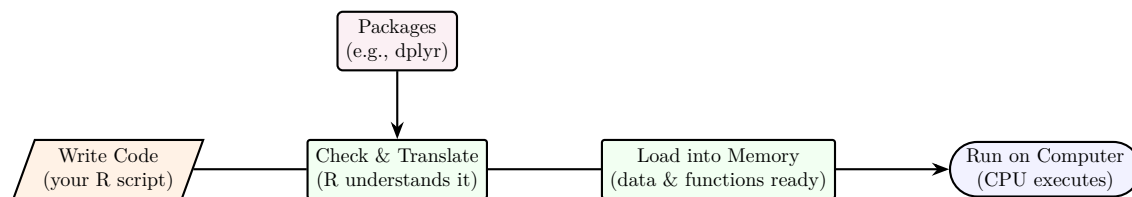
- An effective data handling and storage facility.
- A suite of operators for calculations on arrays, particularly matrices.
- A large, coherent, integrated collection of intermediate tools for data analysis.
- Graphical facilities for data analysis and display, either directly at the computer or on a hard copy.
- A well-developed, simple, and effective programming language (called 'S'), which includes conditionals, loops, user-defined recursive functions, and input and output facilities.

1.4 R and Statistics

R is widely used as a statistical computing system. It provides:

- *Statistics*: While R was not initially introduced as a statistics system, many users adopt it for statistical analysis.
- *Classical and Modern Techniques*: R serves as an environment where both classical and modern statistical methods are implemented.
- *Built-in and Package-based Functionality*: Some statistical functions are built into the base R environment, while many others are supplied through *packages*.
- *Extensive Package Repository*: Around 25 standard and recommended packages come with R, and many more are available through the CRAN network (<http://CRAN.R-project.org>).
- <https://support.posit.co/hc/en-us/articles/201057987-Quick-list-of-useful-R-packages>
- *Comprehensive Methodology*: Most classical statistical methods and the latest methodologies are accessible in R, though users may need to explore available packages to find specific techniques.

How R Code Runs



Basic view: You write R code → R checks and translates it → R prepares everything in memory → the computer runs it. Packages are extra tools you can bring in at the “check & translate” stage.

Live R Demos (using Shiny)

Quick Crowd-Pleasers

- Movie Explorer – filter thousands of films, ratings, and genres.
- Bus Dashboard – real-time style dashboard illustrating inputs, KPIs, and charts.

Public Data Examples

- Animated NYC Metro Traffic – animated ridership through time.
- Crime Watch (UK Police API) – map and bar charts from live police data.
- ScotPHO Profiles Tool – health indicators dashboard.

Fun / Ed-Tech Apps

- Radiant – a full stats/ML environment built in Shiny.
- LEGO Mosaic Creator – upload an image to generate a LEGO mosaic.

1.5 Using R and RStudio Interactively

- **Starting RStudio:** RStudio is an integrated development environment (IDE) for R that provides an easier way to write, debug, and visualize R scripts.

- Download and install R from <https://cran.r-project.org/>.
- Download and install RStudio from <https://posit.co/download/rstudio-desktop/>.
- Open RStudio by clicking the application shortcut.

- **Running R Code in RStudio:**

- The RStudio interface includes four main panels:
 - * **Console:** Directly enter and execute R commands.
 - * **Source Editor:** Write and save scripts (‘.R’ files).
 - * **Environment/History:** View variables and command history.
 - * **Plots/Packages/Help:** Visualize graphs and manage packages.
- You can execute commands by typing them into the console:

```
> x <- c(1, 2, 3, 4, 5)
> mean(x)
```

- To run lines from a script, highlight them and press:

```
Ctrl + Enter (Windows/Linux)
Cmd + Enter (Mac)
```

- **Managing Packages in RStudio:**

- Install packages using:

```
install.packages("ggplot2")
```

- Load an installed package:

```
library(ggplot2)
```

- View installed packages via:

```
Tools > Install Packages
```

- **Quitting RStudio and Saving Work:**

- To quit RStudio, go to:

```
File > Quit Session
```

- RStudio will prompt you to save your workspace and scripts.
- Next time you open the project, it will restore the session.

- **Restarting R in RStudio:**

- To restart the R session without closing RStudio:

```
Session > Restart R
```

- This clears all objects and reloads the session.

1.6 Executing Scripts

- **Running an External R Script:**

- If commands are stored in an external file, such as ‘commands.R’ in the working directory ‘work’, they can be executed at any time in an R session using:

```
source("commands.R")
```

- On Windows, you can also run an external script using:

```
File > Source
```

1.7 Data Permanency and Removing Objects

- **Understanding Objects in R:**

- R creates and manipulates **objects**, which can be variables, arrays, numbers, character strings, functions, or more complex structures.
- During an R session, objects are created and stored by name in what is called the **workspace**.

- **Listing Objects in the Workspace:**

- To display the names of currently stored objects, use:

```
objects()
```

- Alternatively, you can use:

```
ls()
```

- **Removing Objects from the Workspace:**

- To remove objects from the workspace, use the 'rm()' function:

```
rm(x, y, z, ink, junk, temp, foo, bar)
```

- This permanently deletes the specified objects from the session.

- **Saving and Reloading Objects in R:**

- At the end of each R session, you can save all objects for future use.
- If saved, objects are stored in a file called 'RData' in the working directory.
- Command history is also saved in a file called 'Rhistory'.
- When restarting R in the same directory, the workspace is reloaded automatically. Or you can load with:

```
load("name_of_file.RData")
```

- **Best Practices for Workspace Management:**

- It is recommended to use **separate working directories** for different analyses.
- Common object names like 'x' and 'y' should be used cautiously to avoid confusion when switching between different analyses.

1.8 Comments in R

- **Using Comments in R:**

- Comments in R are used to annotate code and improve readability.
- To create a comment, use the hash symbol ('#'), and everything after it on the same line is ignored by the interpreter.

- **Example of a Comment:**

```
# This is a comment in R...
```

- **Inline Comments:**

- Comments can also appear after valid R commands.

```
1 + 1 # This works out the result of one plus one!
```

Output:

```
[1] 2
```

- **Why Use Comments?:**

- Comments help document the code for future reference.
- They improve code readability for both yourself and others.
- They are useful when writing complex scripts or functions.

1.9 Working Directory

- **Understanding the Working Directory:**

- An active R session always has a working directory associated with it.
- Unless a file path is explicitly specified, R uses the working directory by default when saving or importing files.
- To check the current working directory, use:

```
getwd()
```

Output:

```
[1] "/Users/fcakir"
```

- **Changing the Working Directory:**

- File paths must be enclosed in double quotation marks.
- R uses **forward slashes** (/) instead of backslashes (\) for folder paths.
- To change the working directory, use:

```
setwd("/folder1/folder2/folder3/")
```

- **Best Practices for Working Directories:**

- You can specify file paths **relative** to the current working directory or **absolute** paths from the root drive.
- R is **case-sensitive**, so folder names must match exactly.
- If a full and correct file path is provided when reading or writing files, files can reside anywhere on the computer.

1.10 Saving Work and Exiting R

- **Saving Work in R:**

- When finishing an R session, you need to ensure that any objects created and stored in the session, as well as R script files written in an editor, are saved properly.

- **Workspaces:**

- The GUI menu (e.g., under File in Windows or Workspace in macOS) allows saving and loading of workspace image files.
- A workspace image contains all objects in the R session at the time of exit and is saved as an .RData file.
- This file includes:
 - * All assigned objects within the session.
 - * Any objects loaded from a previous workspace file.
- Loading a stored ‘RData’ file allows you to resume your work from where you left off.

- **Listing Workspace Objects:**

- At any time in an R session, you can list all stored objects using:

```
ls()
```

- **Saving and Loading Workspaces Using Commands:**

- R provides the following commands for handling workspace ‘RData’ files:

```
save.image("path/to/workspace.RData") # Save the current
workspace
load("path/to/workspace.RData")       # Load a previously
saved workspace
```

- These functions take a file argument specifying the folder location and name of the ‘RData’ file.

- **Important Considerations:**

- Saving a workspace image does not retain loaded packages.
- When restarting R, you must reload necessary packages using:

```
library(package_name)
```

- **Quick Exit Command:**

- The quickest way to exit R is by entering the following command:

```
q()
```

- **Saving the Workspace on Exit:**

- When exiting, R prompts you with a dialog asking if you’d like to save the workspace image.
- If you choose to save, R does not open a file browser but instead creates (or overwrites) a default ‘RData’ file in the working directory.

1.11 Scripts in R

- **Importance of Saving Scripts:**

- For complex tasks, it is best to work in R's built-in code editor.
- Saving R scripts is as important as saving a workspace, if not more so.

- **Saving and Opening Scripts:**

- Scripts are saved as plain-text files with an '.R' extension.
- This allows operating systems to associate them with R by default.
- To save a script from the built-in editor:
 - * Navigate to File → Save.
- To open a previously saved script:
 - * Navigate to File → Open Script.

- **Scripts vs. Workspaces:**

- You often don't need to save a workspace ('RData') if your script files are saved.
- Running commands from a saved script in a new R console recreates the objects.
- This avoids accidental overwrites in the default workspace.
- Keeping scripts separate makes it easier to manage multiple projects.

- **Additional File Saving Options:**

- R provides options to save individual objects, such as datasets and plot images, to disk.

1.12 Exercise: Using the Help System in R

Objective: Practice writing code in the script editor and sourcing it into the R console while exploring the help system.

Instructions:

1. Open the R script editor and write a command to display the help file for the function `mean`.
2. Use the following command in your script:

```
help('mean') # Opens the documentation for the mean function
```

3. Save the script and source it into the R console.
4. Observe that the help file appears in the bottom-right window of RStudio (or click on the 'Help' tab if not visible).
5. Explore different sections of the help file, especially the examples provided at the end.

Discussion Questions:

- What information is provided in the help file for the `mean` function?
- What other ways can you access function documentation in R?

```
?mean          # Another way to access the help file
??mean         # Searches for "mean" across all help pages
apropos("mean") # Lists functions related to "mean"
```

- Try running an example from the help file by using:

```
example(mean) # Runs example usage of the mean function from the
              documentation
```

Bonus Challenge:

- Find help documentation for any function related to data manipulation (e.g., `sum`, `median`, or `sd`).

```
help('sum') # Displays documentation for the sum function
help('median') # Displays documentation for the median function
help('sd') # Displays documentation for the standard deviation function
```

- Identify the arguments of the function and explain how they work.

1.13 Exercise: Creating and Assigning Variables in R

Objective: Practice creating variables and assigning values in R while exploring the RStudio Environment tab.

Instructions:

1. Create a numeric variable named `first_num` and assign it the value 42.

```
first_num <- 42
```

2. Click on the Environment tab (top-right window in RStudio) to check if the variable appears with its assigned value.
3. Create another variable called `first_char` and assign it a character string.

```
first_char <- "my first character"
```

4. Verify that the variable `first_char` appears in the Environment tab, displaying both its value and its class (chr for character).

Discussion Questions:

- What happens when you assign a number without quotes?
- How does R differentiate between numeric and character variables?
- What happens if you reassign `first_num` to a character value?

```
first_num <- "Now I am text"
```

- Try running the following command:

```
class(first_num)
class(first_char)
```

Bonus Challenge:

- Create a logical (Boolean) variable and check its class in the Environment tab.

```
my_bool <- TRUE
class(my_bool)
```

1.14 Exercise: Removing Variables from the Environment

Objective: Practice removing variables from the R environment and verifying their removal.

Instructions:

1. Remove the variable `first_num` from your environment using the `rm()` function.

```
rm(first_num) # Removes first_num from the environment
```

2. Check the Environment tab to confirm that `first_num` has been removed.
3. Alternatively, use the `ls()` function to list all objects currently in your environment.

```
ls() # Lists all objects in the environment
```

Discussion Questions:

- What happens when you try to print `first_num` after removing it?

```
print(first_num)
```

- What command would you use to remove multiple variables at once?

```
rm(var1, var2, var3) # Removes multiple variables
```

- How can you remove all objects from the environment at once?

```
rm(list = ls()) # Clears all variables from the environment
```

Bonus Challenge:

- Try removing all objects and checking if the environment is completely empty.

```
rm(list = ls())  
ls()
```